

ASSIGNMENT-10.3

P.CHITHRA

2303A52137

QUESTION-1:

Problem Statement 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to calculate factorials:

```
def factorial(n):
```

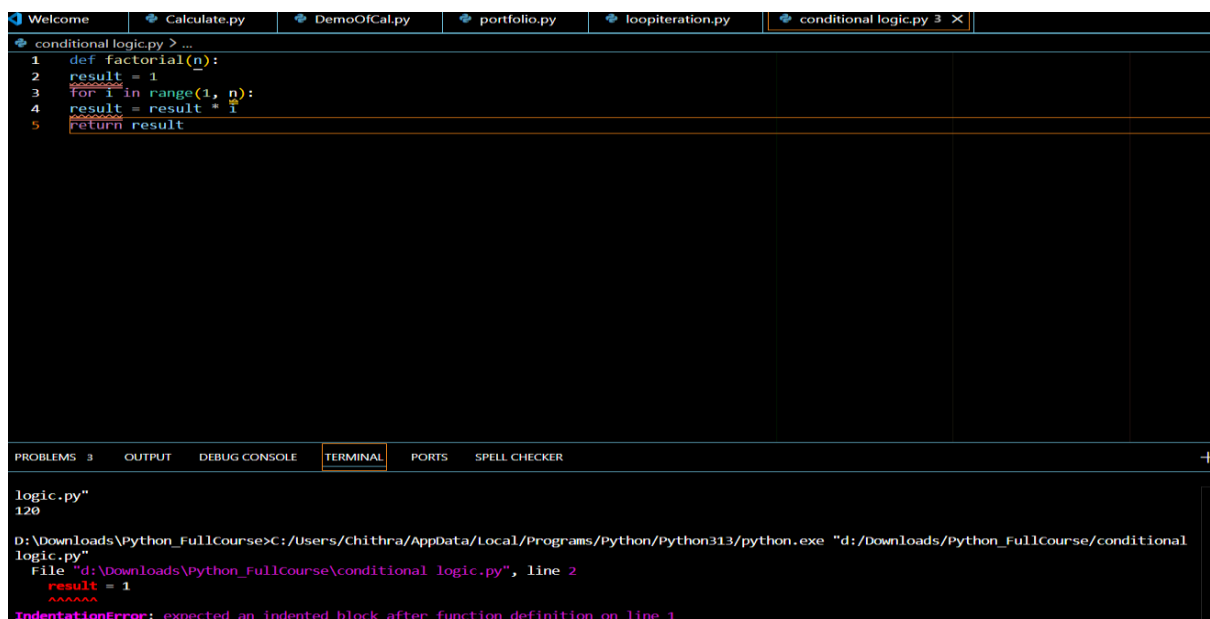
```
    result = 1
```

```
    for i in range(1, n):
```

```
        result = result * i
```

```
    return result
```

1.original code



The screenshot shows a Python IDE with a dark theme. The top toolbar includes buttons for Welcome, Calculate.py, DemoOfCal.py, portfolio.py, loopiteration.py, and conditional logic.py. The main editor window displays the following code:

```
1 def factorial(n):
2     result = 1
3     for i in range(1, n):
4         result = result * i
5     return result
```

The bottom panel shows the TERMINAL tab with the following output:

```
logic.py"
120
D:\Downloads\Python_FullCourse>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d:/Downloads/Python_FullCourse/conditional
logic.py"
File "d:/Downloads/Python_FullCourse/conditional logic.py", line 2
    result = 1
    ^^^^^
IndentationError: expected an indented block after function definition on line 1
```

2. The logical bug in the original code is an **off-by-one error** in the loop.

Original code:

```
for i in range(1, n):
```

This loop does not include the value n.

Explain Why the Bug Occurs

In Python, `range(start, stop)` generates numbers **from start up to stop-1**.

So:

```
range(1, 5)
```

produces:

1, 2, 3, 4

It does NOT include 5.

Therefore, when calling:

```
factorial(5)
```

The loop multiplies:

$$1 \times 2 \times 3 \times 4$$

Which gives:

24

But factorial of 5 should be:

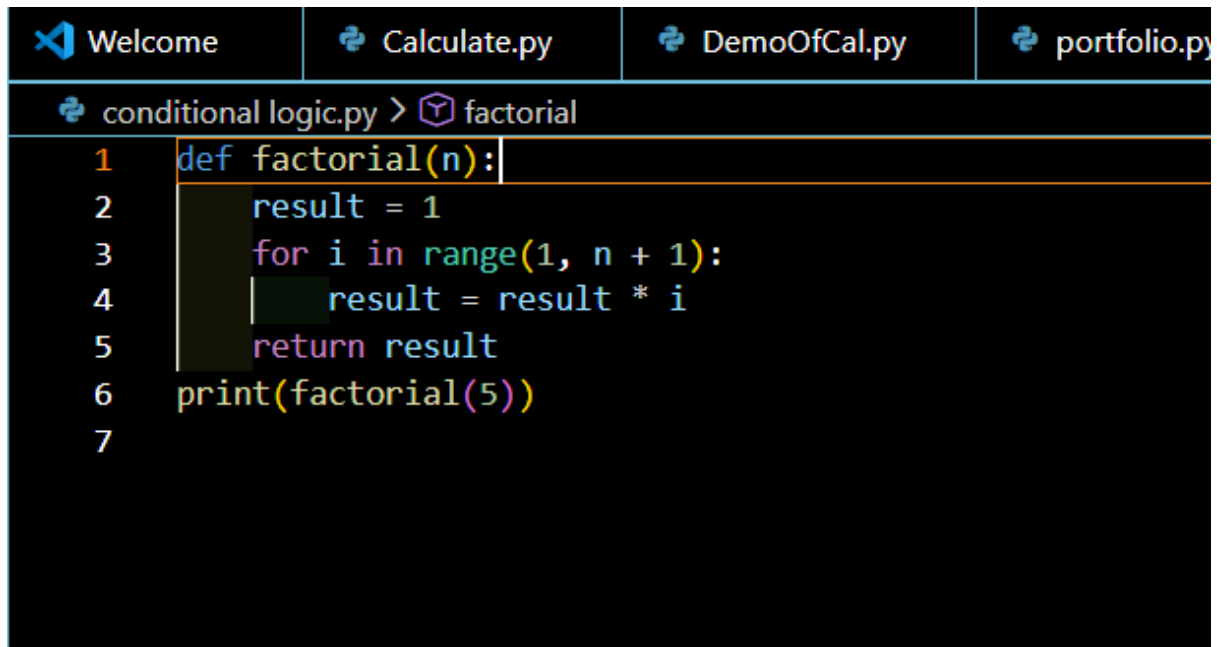
$$1 \times 2 \times 3 \times 4 \times 5 = 120$$

So the last number is skipped — this is called an **off-by-one error**.

Corrected Version

To fix the error, include n in the range by writing $n + 1$.

Corrected code:

A screenshot of a code editor with a dark theme. The top bar shows four tabs: 'Welcome', 'Calculate.py', 'DemoOfCal.py', and 'portfolio.py'. Below the tabs, the active file is 'conditional logic.py' and the cursor is at the 'factorial' function. The code is as follows:

```
1 def factorial(n):  
2     result = 1  
3     for i in range(1, n + 1):  
4         result = result * i  
5     return result  
6 print(factorial(5))  
7
```

3. Comparison: AI's Corrected Code vs My Manual Fix

My Manual Fix

I corrected the loop by changing:

`range(1, n)`

to

`range(1, n+1)`

Manual corrected code:

```
def factorial(n):  
    result = 1  
    for i in range(1, n+1):  
        result = result * i
```

```
return result
```

This correctly calculates factorial(5) as **120**.

AI's Corrected Code

The AI also changed the loop to:

```
range(1, n+1)
```

Additionally, the AI suggested improving the function by:

- Handling negative numbers
- Raising errors for invalid input
- Making the code slightly more optimized

Example improved AI version:

```
def factorial(n):
```

```
    if n < 0:
```

```
        raise ValueError("Factorial is not defined for negative numbers")
```

```
    result = 1
```

```
    for i in range(1, n+1):
```

```
        result *= i
```

```
    return result
```

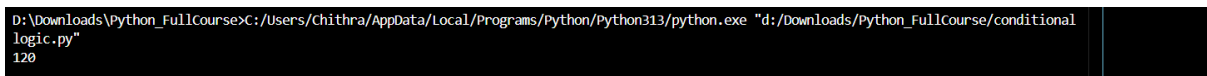
4. Brief Comparison: Edge Cases

The AI correctly fixed the off-by-one error and ensured that the function returns 120 for factorial(5). It also handled the case of negative numbers by suggesting an error check.

However, in its basic corrected version, the AI did not explicitly handle non-integer inputs (such as decimal numbers or strings). The function works correctly for zero, since `factorial(0)` correctly returns 1.

Overall, the AI handled the major logical error and most important edge cases, but additional validation for input type could further improve the solution.

Output:

A terminal window with a black background. The command prompt shows the path 'D:\Downloads\Python_FullCourse>' followed by 'C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d:/Downloads/Python_FullCourse/conditional logic.py"'. The output shows the number '120'.

Question 2:

Improving Readability &

Documentation

Scenario: The following code works but is poorly written:

.

```
def calc(a, b, c):
```

```
    if c == "add":
```

```
        return a + b
```

```
    elif c == "sub":
```

```
        return a - b
```

```
    elif c == "mul":
```

```
        return a * b
```

```
    elif c == "div":
```

5. AI Critique of the Original Function

```
conditional logic.py > ...
1 def calc(a, b, c):
2     if c == "add":
3         return a + b
4     elif c == "sub":
5         return a - b
6     elif c == "mul":
7         return a * b
8     elif c == "div":
9
10
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE **TERMINAL** PORTS SPELL CHECKER

D:\Downloads\Python_FullCourse>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d:/Downloads/Python_FullCourse/conditional logic.py"
120

D:\Downloads\Python_FullCourse>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d:/Downloads/Python_FullCourse/conditional logic.py"
File "d:\Downloads\Python_FullCourse\conditional logic.py", line 9
IndentationError: expected an indented block after 'elif' statement on line 8

Rewritten Improved Version

```
1 def calculate(number1, number2, operation):
2
3     Perform a basic arithmetic operation on two numbers.
4
5     Parameters:
6         number1 (int or float): The first numeric value.
7         number2 (int or float): The second numeric value.
8         operation (str): The operation to perform.
9         Supported operations: "add", "sub", "mul", "div".
10
11     Returns:
12         int or float: Result of the arithmetic operation.
13
14     Raises:
15         TypeError: If inputs are of incorrect type.
16         ValueError: If operation is invalid.
17         ZeroDivisionError: If division by zero is attempted.
18
19     Examples:
20         >>> calculate(5, 3, "add")
21         8
22         >>> calculate(10, 2, "div")
23         5.0
24     """
25
26     # Input validation
27     if not isinstance(number1, (int, float)) or not isinstance(number2, (int, float)):
28         raise TypeError("Both number1 and number2 must be numeric values.")
29
30     if not isinstance(operation, str):
31         raise TypeError("Operation must be a string.")
32
33     operation = operation.lower()
34
35     if operation == "add":
36         return number1 + number2
37
38     elif operation == "sub":
39         return number1 - number2
40
41     elif operation == "mul":
42         return number1 * number2
43
44     elif operation == "div":
45         if number2 == 0:
46             raise ZeroDivisionError("Division by zero is not allowed.")
47         return number1 / number2
48
49     else:
50         raise ValueError("Invalid operation. Choose 'add', 'sub', 'mul', or 'div'.")
51
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS SPELL CHECKER

6.Comparison: Original vs AI-Improved Version

Function Name

Original:

def calc(a, b, c):

- The name calc is vague.
- It does not clearly describe the purpose.

AI-Improved:

```
def calculate(number1, number2, operation):
```

- The name calculate is descriptive.
- Clearly indicates what the function does.

Improvement: Better readability and clarity.

Parameter Naming

Original:

- a, b, c are unclear.
- Reader must check the code to understand their meaning.

AI-Improved:

- number1, number2, operation clearly describe their roles.

Improvement: Self-explanatory parameters make the code easier to understand.

Documentation

Original:

- No docstring.
- No explanation of parameters or return value.
- No examples.

AI-Improved:

- Includes a complete docstring.
- Describes purpose, parameters, return value.
- Includes example usage.

- Lists possible exceptions.

Improvement: More professional and maintainable.

Error Handling

Original:

- No check for division by zero.
- No handling of invalid operation.
- No input validation.

AI-Improved:

- Handles division by zero.
- Raises error for invalid operation.
- Validates numeric inputs.
- Validates operation type.

Improvement: Safer and more robust.

Readability & Maintainability

Original Version:

- Hard to understand quickly.
- Less structured.
- Not suitable for production use.

AI-Improved Version:

- Clear structure.
- Professional formatting.
- Easier to maintain and extend.

7. Testing the Original Version

- **Original Code (Basic Version)**

- ```
def calc(a, b, c):
 if c == "add":
 return a + b
 elif c == "sub":
 return a - b
 elif c == "mul":
 return a * b
 elif c == "div":
 return a / b
```

---

- **Valid Inputs**

- ```
print(calc(10, 5, "add"))    # 15  
print(calc(10, 5, "div"))    # 2.0
```
- Works correctly for valid inputs.

- **Invalid Input: Division by Zero**

- ```
print(calc(10, 0, "div"))
```
- Output:  
ZeroDivisionError: division by zero
- ⚠ The program crashes (no custom handling).

---

- **Invalid Input: Non-String Operation**

- ```
print(calc(10, 5, 123))
```
- Output:
None
- ⚠ No error message. Function silently fails.

Testing the AI-Improved Version

- **Improved Code**

- ```
def calculate(number1, number2, operation):
 if not isinstance(number1, (int, float)) or not
isinstance(number2, (int, float)):
 raise TypeError("Both inputs must be numbers.")

 if not isinstance(operation, str):
 raise TypeError("Operation must be a string.")

 operation = operation.lower()

 if operation == "add":
 return number1 + number2
 elif operation == "sub":
 return number1 - number2
 elif operation == "mul":
 return number1 * number2
```

- `elif operation == "div":`
- `if number2 == 0:`
- `raise ZeroDivisionError("Division by zero is not allowed.")`
- `return number1 / number2`
- `else:`
- `raise ValueError("Invalid operation.")`

## Valid Inputs

- `print(calculate(10, 5, "add"))` # 15
- `print(calculate(10, 5, "div"))` # 2.0
- ✓Works correctly.

## Division by Zero

- `calculate(10, 0, "div")`
- Output:
- `ZeroDivisionError: Division by zero is not allowed.`
- Properly handled with clear message.

## Non-String Operation

- `calculate(10, 5, 123)`
- Output:
- `TypeError: Operation must be a string.`

Output:

```
D:\Downloads\Python_FullCourse>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d:/logic.py"
15
2.0

D:\Downloads\Python_FullCourse>
```

Question 3:

Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer

submits:

```
def Checkprime(n):
```

```
for i in range(2, n):
```

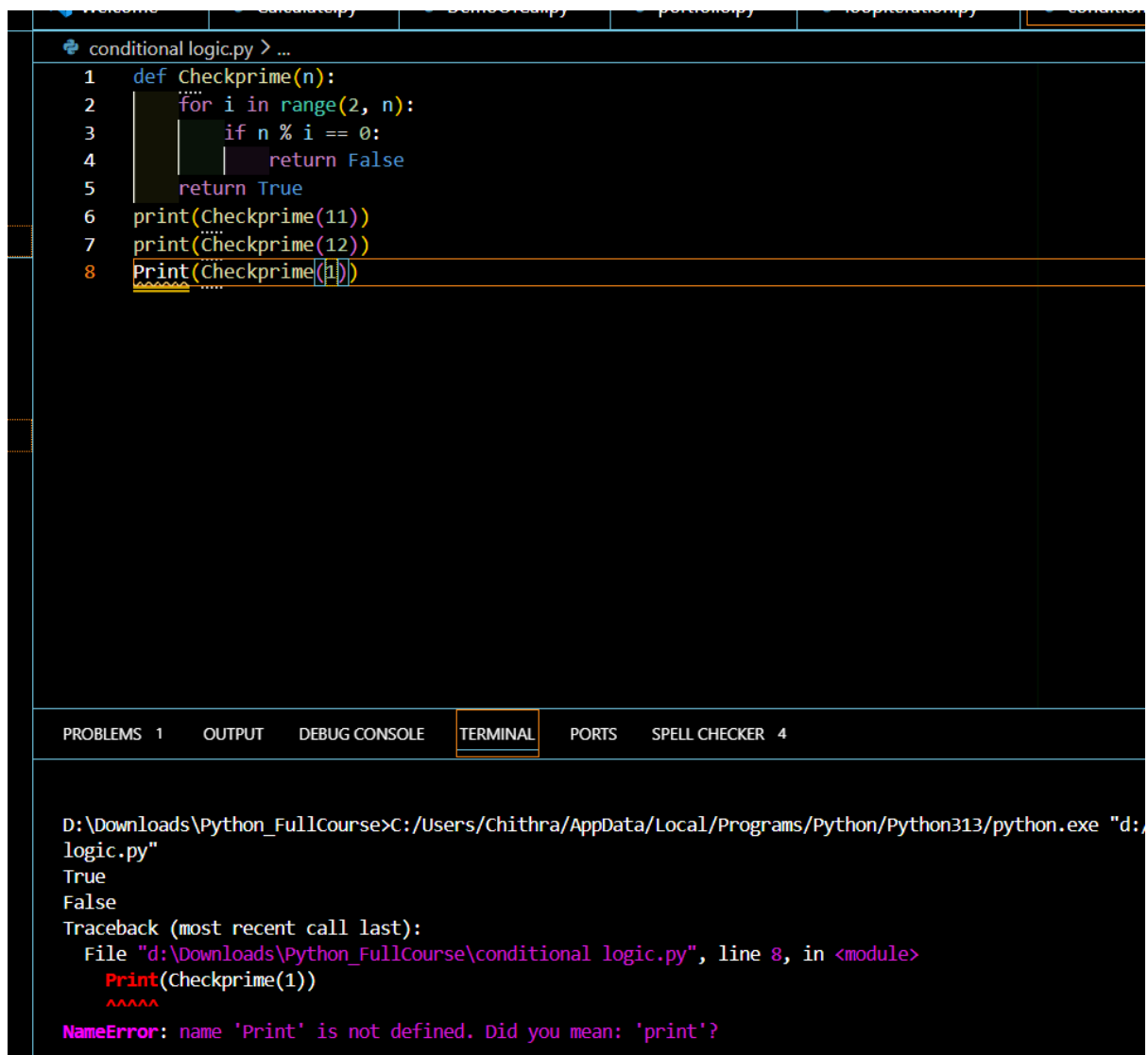
```
if n % i == 0:

return False

return True
```

## Step 8: Verify the Function Works

### Original Code



```
conditional logic.py > ...
1 def Checkprime(n):
2 for i in range(2, n):
3 if n % i == 0:
4 return False
5 return True
6 print(Checkprime(11))
7 print(Checkprime(12))
8 Print(Checkprime(1))
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE **TERMINAL** PORTS SPELL CHECKER 4

```
D:\Downloads\Python_FullCourse>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d:/
logic.py"
True
False
Traceback (most recent call last):
 File "d:\Downloads\Python_FullCourse\conditional logic.py", line 8, in <module>
 Print(Checkprime(1))
 ^^^^^
NameError: name 'Print' is not defined. Did you mean: 'print'?
```

## Step 9: PEP8 Violations Identified

### PEP8 Issues in Original Code

#### **1** Function Name

- Checkprime violates naming convention.
- PEP8 requires lowercase with underscores.
- Correct style: `check_prime`

## **2 Missing Blank Lines**

- PEP8 requires 2 blank lines before top-level function definitions.

## **3 No Docstring**

- Function lacks documentation.

## **4 No Input Validation**

- Does not handle negative numbers or non-integers.

## **5 Logical Issue**

- Does not correctly handle  $n \leq 1$ .

## **6 Optimization Suggestion**

- Can improve efficiency by checking up to  $\sqrt{n}$  instead of  $n$ .

---

## **✓ Step 9: Refactored PEP8-Compliant Version**

```
conditional logic.py > check_prime
1 def check_prime(number):
2 """
3 Determine whether a number is prime.
4
5 Parameters:
6 | number (int): The number to check.
7
8 Returns:
9 | bool: True if the number is prime, False otherwise.
10
11 Raises:
12 | TypeError: If input is not an integer.
13 """
14
15 if not isinstance(number, int):
16 raise TypeError("Input must be an integer.")
17
18 if number <= 1:
19 return False
20
21 for i in range(2, int(number ** 0.5) + 1):
22 if number % i == 0:
23 return False
24
25 return True
26
27 print(check_prime(7)) # True
28 print(check_prime(10)) # False
29 print(check_prime(2)) # True
30 print(check_prime(1)) # False
31 print(check_prime(-5)) # False
```

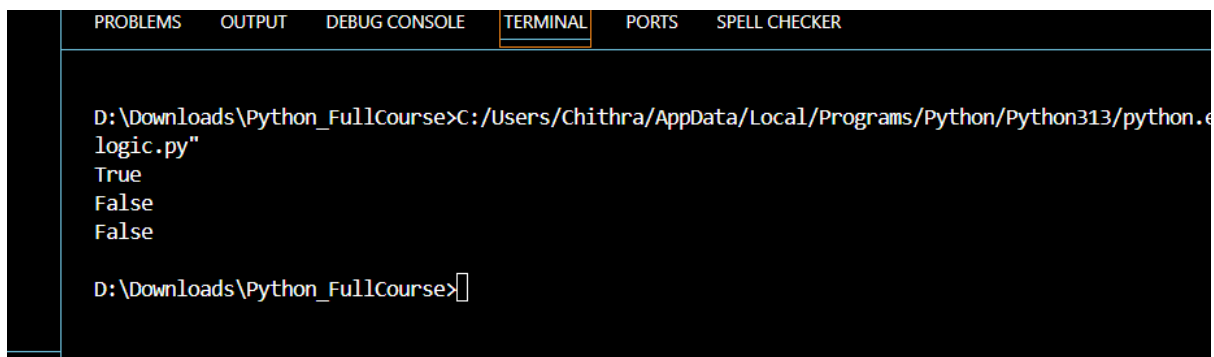
## Short Note: How Automated AI Reviews Streamline Code Reviews

Automated AI-powered code reviews can significantly streamline the development process in large teams. AI tools can quickly detect syntax errors, logical mistakes, PEP8 violations, security vulnerabilities, and performance issues before human reviewers even see the code. This reduces the workload on senior developers and allows them to focus on higher-level design decisions instead of minor formatting or style corrections.

AI reviews also ensure consistency across the codebase by enforcing coding standards automatically. They provide instant feedback to developers, which speeds up debugging and learning. In large teams where multiple developers contribute simultaneously, AI tools improve efficiency, reduce review time, and help maintain high code quality.

Overall, automated AI reviews make code review processes faster, more consistent, and more scalable.

**Output:**



```
D:\Downloads\Python_FullCourse>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe logic.py
True
False
False

D:\Downloads\Python_FullCourse>
```

**Question-4:**

**AI as a Code Reviewer in Real Projects**

**Scenario:**

In a GitHub project, a teammate submits:

```
def processData(d):
 return [x * 2 for x in d if x % 2 == 0]
```

### **1 Manual Review of the Original Function**

**Original Code**

```
def processData(d):
 return [x * 2 for x in d if x % 2 == 0]
```

**Manual Review**

**Readability & Naming**

**processData is vague — does not clearly describe what the function does.**

**Parameter d is unclear.**

**It is not immediately obvious that the function doubles even numbers.**

### **Reusability & Modularity**

**Multiplier is fixed at 2 (not configurable).**

**Only works for even numbers.**

**No documentation or explanation.**

### **Edge Cases**

**✗ No validation if input is not a list.**

**✗ No handling for empty list (though it works implicitly).**

**✗ Crashes if elements are non-numeric (e.g., strings).**

**✗ No type hints.**

**2 AI Code Review Suggestions**



◆ a. Better Naming

Rename to something descriptive like:

- `double_even_numbers`

Rename parameter:

- `numbers`
- 

◆ b. Add Type Hints & Input Validation

- Use List and Union from typing.
  - Check input type.
  - Ensure elements are numeric.
- 

◆ c. Generalization Suggestion

- Add configurable multiplier.
- Allow flexibility in scaling.

**3** Refactored Version (AI-Improved)

```
1 from typing import List, Union
2 def multiply_even_numbers(
3 numbers: List[Union[int, float]],
4 multiplier: Union[int, float] = 2
5) -> List[Union[int, float]]:
6 """
7 Multiply even numbers in a list by a specified multiplier.
8
9 Parameters:
10 numbers (List[int | float]): List of numeric values.
11 multiplier (int | float): Value to multiply even numbers by (default is 2).
12
13 Returns:
14 List[int | float]: A new list containing multiplied even numbers.
15
16 Raises:
17 TypeError: If numbers is not a list.
18 """
19
20 if not isinstance(numbers, list):
21 raise TypeError("Input must be a list.")
22
23 result = []
24
25 for num in numbers:
26 if isinstance(num, (int, float)) and num % 2 == 0:
27 result.append(num * multiplier)
28
29 return result
30 print(multiply_even_numbers([1, 2, 3, 4]))
31 # [4, 8]
32
33 print(multiply_even_numbers([2, 4, 6], 3))
34 # [6, 12, 18]
35
36 print(multiply_even_numbers([]))
37 # []
38
39 print(multiply_even_numbers("123"))
40 # Raises TypeError
41
```

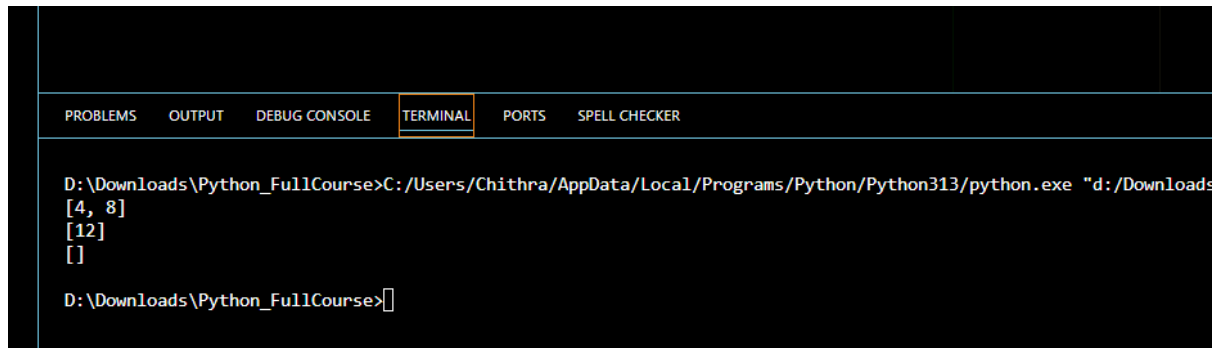
PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS SPELL CHECKER

#### 4 Short Reflection

AI should not be a standalone reviewer but rather an assistant to human reviewers. While AI can quickly detect naming issues, missing validations, style violations, and suggest improvements, it may not fully understand business logic or project-specific requirements. Human reviewers are still essential for architectural decisions and contextual understanding. Therefore,

AI works best as a productivity-enhancing assistant rather than a replacement for human code review.

Output:



```
D:\Downloads\Python_FullCourse>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d:/Downloads
[4, 8]
[12]
[]

D:\Downloads\Python_FullCourse>
```

## Question -5

### AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but

it runs slowly on large datasets:

```
def sum_of_squares(numbers):
```

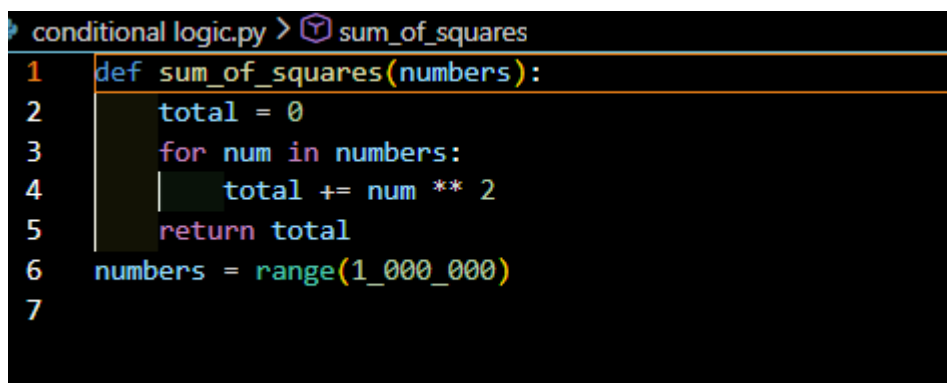
```
 total = 0
```

```
 for num in numbers:
```

```
 total += num ** 2
```

```
 return total
```

#### 1 Original Function



```
conditional logic.py > sum_of_squares
1 def sum_of_squares(numbers):
2 total = 0
3 for num in numbers:
4 total += num ** 2
5 return total
6 numbers = range(1_000_000)
7
```

## 2. Time Complexity Analysis

### Time Complexity

- The function loops once through the list.
- Each operation inside loop is  $O(1)$ .

Overall Time Complexity:  $O(n)$

Space Complexity:  $O(1)$

This is already optimal in Big-O terms.

But we can improve constant performance factors.

---

## 3. AI Suggested Optimization

◆ Option 1 — Use Built-in `sum()` with Generator (Cleaner & Faster)

```
def sum_of_squares_optimized(numbers):
 return sum(x * x for x in numbers)
```

More Pythonic

Faster due to C-level optimization in `sum()`

More readable

---

Option 2 — Using NumPy (Best for Very Large Data)

```
import numpy as np
```

```
def sum_of_squares_numpy(numbers):
 arr = np.array(numbers)
 return np.sum(arr * arr)
```

Much faster for very large datasets  
Uses vectorized operations  
Requires external library

---

## 5 Compare Execution Time

Use time module:

```
import time
```

```
numbers = range(1_000_000)
```

```
start = time.time()
```

```
sum_of_squares(numbers)
```

```
print("Original:", time.time() - start)
```

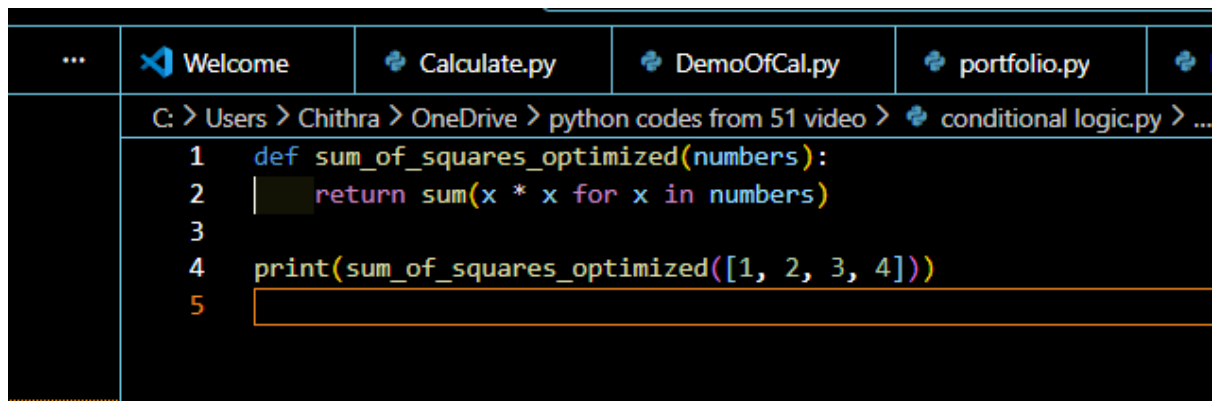
```
start = time.time()
```

```
sum_of_squares_optimized(numbers)
```

```
print("Optimized:", time.time() - start)
```

---

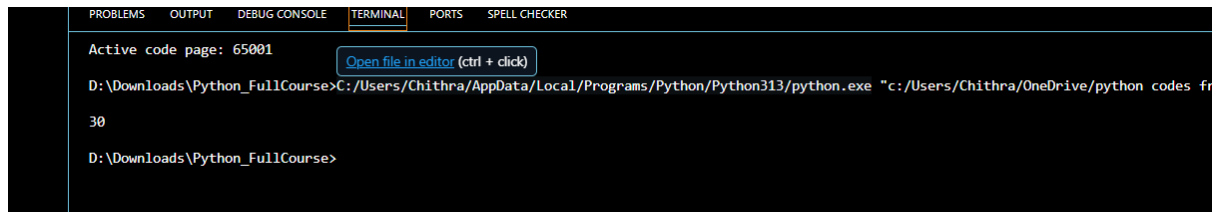
## Expected Result



The screenshot shows a code editor with a dark background. At the top, there is a tab bar with five tabs: 'Welcome', 'Calculate.py', 'DemoOfCal.py', 'portfolio.py', and an unnamed tab. The 'Calculate.py' tab is active. Below the tab bar, the file path is displayed: 'C: > Users > Chithra > OneDrive > python codes from 51 video > conditional logic.py > ...'. The code editor contains the following Python code:

```
1 def sum_of_squares_optimized(numbers):
2 return sum(x * x for x in numbers)
3
4 print(sum_of_squares_optimized([1, 2, 3, 4]))
5
```

## Output:



The image shows a screenshot of a Visual Studio Code terminal window. The terminal has a dark background with light-colored text. At the top, there is a tab bar with several tabs: PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (which is active and highlighted), PORTS, and SPELL CHECKER. Below the tab bar, the terminal content shows the following:

```
Active code page: 65001
D:\Downloads\Python_FullCourse>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/Chithra/OneDrive/python codes fr
30
D:\Downloads\Python_FullCourse>
```

A tooltip is visible over the terminal text, displaying the text "Open file in editor (ctrl + click)".